

Bàn về tư tưởng tương tự

Zhou Gelin

Trường Trung học Changjun, Trường Sa

2006

Nguồn gốc: Bàn về tư tưởng tương tự

IOI China candidate team papers / 2006 / Zhou Gelin.doc

Tóm tắt

Tin học là một nghệ thuật luôn biến hóa, chứa rất nhiều mảng kiến thức. Ta không thể mong nắm hết mọi tri thức, mà cần dựa trên kiến thức đã có để chuyển những bài toán lạ thành bài toán quen. Tư tưởng tương tự là một phương pháp chuyển hóa rất tốt. Bài viết thử giải thích một số mẫu tương tự thường gặp trong thi tin học.

Từ khóa: tư tưởng tương tự; mô hình; thuật toán; nhận thức lý tính.

1 Dẫn nhập

Tương tự là một phương pháp tư duy giàu sức sáng tạo. Nó chú ý tới việc hai đối tượng giống hoặc gần giống nhau ở một số phương diện, rồi suy đoán rằng chúng cũng có thể giống nhau ở những phương diện khác.

Một phép tương tự đơn giản có dạng:

đối tượng A có tính chất P, Q ,
đối tượng A' có tính chất P' tương tự P ,
vì vậy A' có thể có tính chất Q' tương tự Q .

Ví dụ, cách cầm bút chì có thể được chuyển sang bút máy vì hai loại bút đều là bút cứng, thân tương đối nhỏ. Nhưng cách ấy thất bại với bút lông, vì bút lông là bút mềm và cách điều khiển đầu bút khác hẳn. Phép tương tự thành công không chỉ nhìn vào vài tính chất bề ngoài, mà còn phải xét quan hệ giữa các tính chất ấy.

Bài viết gọi phép tương tự dựa chủ yếu vào trực giác là **tương tự đơn giản**; còn phép tương tự có xét quan hệ bản chất giữa các tính chất là **tương tự khoa học**. Trong thi tin học, ta cần loại thứ hai. Các phần sau trình bày ba mẫu thường gặp:

- lấy sự vật cụ thể tương tự với mô hình trừu tượng;
- tương tự giữa các thuật toán gần nhau;
- chuyển hình học sang biểu thức số học.

2 Sự vật cụ thể và mô hình trừu tượng

Đây là dạng tương tự thường gặp nhất. Sự vật thực tế không phải mô hình toán học nghiêm ngặt; khi nghiên cứu, ta phải trích ra mô hình phù hợp. Nếu mô hình chọn sai góc nhìn, bài toán có thể trở nên khó không cần thiết.

2.1 Ví dụ: bài toán đỉnh núi

Bài toán mô tả một nông trại hẹp, chia thành n mảnh đất trên một đường thẳng. Mỗi mảnh có độ cao. Một đoạn liên tiếp cùng độ cao được gọi là **đỉnh núi** nếu hai bên của nó đều thấp hơn hoặc là biên. Ta chỉ được giảm độ cao, không được tăng; cần làm cho số đỉnh núi không vượt quá k , với tổng lượng đất bị dời đi nhỏ nhất.

Ấn tượng đầu tiên là một bài quy hoạch động tuyến tính. Chẳng hạn, đặt $f(i, j)$ là chi phí nhỏ nhất để xử lý i mảnh đầu và còn j đỉnh núi. Nhưng trạng thái này không đủ mô tả ảnh hưởng của độ cao hiện tại tới phần sau. Muốn chính xác lại phải lưu một tập dãy độ cao, chi phí lưu trữ không thể chấp nhận.

Ta đổi góc nhìn: lật ngược toàn bộ địa hình, rồi chia đất theo các tầng độ cao. Mỗi tầng liên tiếp cùng độ cao được xem như một nút. Tầng cao hơn được tầng thấp hơn chống đỡ; nối quan hệ chống đỡ ấy thành cạnh. Khi đó cấu trúc thu được là một cây có gốc:

- một tầng có thể chống đỡ nhiều tầng phía trên;
- từ gốc, mỗi tầng chỉ được chống đỡ bởi đúng một tầng thấp hơn;
- các đỉnh núi chính là các nút lá, vì chúng không chống đỡ tầng nào khác.

Việc dời đất tương ứng với xóa nút; lượng đất dời đi là trọng số của nút. Bài toán trở thành: cho một cây có gốc, mỗi nút có trọng số, xóa một số nút sao cho số lá còn lại không vượt quá K , và tổng trọng số xóa là nhỏ nhất. Gốc không được tính là lá. Đây là một bài quy hoạch động trên cây kinh điển, có thể giải trong $\mathcal{O}(nk^2)$.

Phần còn lại là dựng cây hiệu quả. Quét các mảnh đất từ trái sang phải và duy trì một ngăn xếp các đoạn cùng độ cao:

- nếu độ cao mới lớn hơn tầng trên cùng, đưa tầng mới vào ngăn xếp;
- nếu bằng, kéo dài tầng trên cùng;
- nếu nhỏ hơn, các tầng cao hơn đã kết thúc, lần lượt lấy ra để tạo nút và nối với tầng chống đỡ.

Mỗi tầng vào và ra ngăn xếp nhiều nhất một lần, nên dựng cây mất $\mathcal{O}(n)$.

Điểm chính của ví dụ là mô hình tuyến tính ban đầu không giúp nhiều, còn phép tương tự giữa “tầng đất” và “nút cây” làm bài toán trở thành một dạng quen thuộc.

3 Tương tự giữa các thuật toán

Một số thuật toán giống nhau về tư tưởng, căn cứ đúng đắn, hoặc cách hiện thực. Nhờ tương tự, ta có thể cải tạo một thuật toán bằng kỹ thuật của thuật toán khác.

3.1 Ví dụ: cực tiểu hóa cạnh lớn nhất trong nhiều đường đi

Cho n thành phố và p đường hai chiều. Cần tìm t đường đi từ thành phố 1 tới thành phố n , đôi một không dùng chung cạnh. Trong các phương án như vậy, cần cực tiểu hóa độ dài cạnh lớn nhất xuất hiện trên các

đường đi.

Mô hình luồng rất tự nhiên: mỗi cạnh có dung lượng 1, cần luồng ít nhất t , và mục tiêu là cực tiểu hóa cạnh lớn nhất được dùng. Cách đơn giản là nhị phân đáp án m , bỏ mọi cạnh dài hơn m , rồi kiểm tra có luồng ít nhất t hay không. Nếu dùng thuật toán tăng luồng cơ bản, độ phức tạp vào khoảng $\mathcal{O}(tp \log p)$.

Ta so sánh bài này với hai cặp bài quen thuộc:

- Min-cost max-flow cực tiểu hóa tổng chi phí cạnh được dùng.
- Minimum spanning tree có thể xem như cực tiểu hóa cạnh lớn nhất theo tư tưởng giống bài “minimax”, trong khi shortest path cực tiểu hóa tổng.

Trong min-cost max-flow, ta liên tục tìm đường tăng có chi phí nhỏ nhất. Tương tự, trong bài này ta tìm đường tăng sao cho cạnh lớn nhất trên đường là nhỏ nhất. Gán nhãn tạm thời

$$dist[v] = \min \text{ giá trị cạnh lớn nhất trên một đường tăng tới } v.$$

Khi xét cạnh chưa dùng (u, v) , cập nhật

$$dist[v] \leftarrow \min\{dist[v], \max(dist[u], w(u, v))\}.$$

Nếu có cạnh ngược trong mạng dư tương ứng với việc hủy một phần luồng cũ, việc đi qua cạnh ngược không làm tăng giá trị cạnh lớn nhất của đường tăng, nên có thể cập nhật bằng $dist[u]$.

Bản gốc chứng minh bằng hai ý:

1. Giá trị nhãn của đích trên các đường tăng được chọn là không giảm.
2. Việc đi qua cạnh ngược không làm giảm giá trị cạnh lớn nhất của luồng hiện tại; nếu nó làm giảm được thì các đường tốt hơn đã phải được chọn sớm hơn.

Do đó mỗi lần tăng luồng theo đường minimax vẫn giữ nghiệm tối ưu cho lượng luồng hiện tại. Việc tính nhãn có thể dùng tư tưởng giống Prim. Với cài đặt đơn giản, mỗi lần tìm đường tốn $\mathcal{O}(n^2 + p)$, tổng cộng $\mathcal{O}(t(n^2 + p))$. Với đồ thị thưa, có thể dùng heap để giảm chi phí.

Giá trị của ví dụ nằm ở phép tương tự: từ “tổng chi phí nhỏ nhất” trong min-cost flow chuyển sang “cạnh lớn nhất nhỏ nhất”, giống sự khác nhau giữa shortest path và minimum spanning tree.

4 Hình học và biểu thức số

Về trực giác, hình học và biểu thức số rất khác nhau. Nhưng đôi khi bỏ qua thông tin hình học thứ yếu và giữ lại đại lượng bất biến lại làm bài toán dễ hơn. Mấu chốt là phân biệt đâu là thông tin cần cho lời giải.

4.1 Ví dụ: đẳng cấu tập điểm

Cho hai tập S và S' , mỗi tập gồm n điểm. Cần xác định liệu chúng có thể trùng nhau sau các phép tịnh tiến, quay, lật và co giãn đồng dạng hay không.

Tịnh tiến được xử lý bằng cách xét trọng tâm. Nếu S có các điểm (x_i, y_i) , trọng tâm là

$$O = \left(\frac{1}{n} \sum_{i=1}^n x_i, \frac{1}{n} \sum_{i=1}^n y_i \right).$$

Sau khi chuyển sang hệ tọa độ lấy trọng tâm làm gốc, phép tịnh tiến không còn ảnh hưởng.

Co giãn được xử lý bằng tỉ lệ khoảng cách. Nếu m là khoảng cách lớn nhất từ trọng tâm tới một điểm trong S , và m' là giá trị tương ứng của S' , thì tỉ lệ co giãn là $k = m'/m$ nếu $m \neq 0$. Nhờ đó có thể chuẩn hóa các khoảng cách.

Vấn đề còn lại là quay. Với mỗi điểm P , biểu diễn nó bằng cặp:

$$(\alpha_P, |OP|),$$

trong đó α_P là góc phương vị quanh trọng tâm. Sắp xếp các điểm theo góc, nếu bằng góc thì theo khoảng cách. Thay vì lưu góc tuyệt đối, lưu hiệu góc giữa hai điểm kề nhau trong thứ tự vòng tròn, cùng với khoảng cách đã chuẩn hóa. Khi đó một tập điểm trở thành một chuỗi vòng gồm các phần tử

$$(\Delta\alpha_i, d_i).$$

Quay toàn bộ hình chỉ làm thay đổi vị trí bắt đầu của chuỗi vòng. Vì vậy hai tập điểm đẳng cấu nếu chuỗi vòng của một tập xuất hiện trong chuỗi nhân đôi của tập kia. Bài toán hình học chuyển thành bài toán so khớp chuỗi vòng, có thể dùng KMP.

Lập hình được xử lý bằng cách đảo thứ tự góc và kiểm tra thêm một lần. Tổng chi phí bị chi phối bởi bước sắp xếp:

$$\mathcal{O}(n \log n).$$

Trường hợp điểm trùng trọng tâm cần được đặc biệt xử lý, vì góc của nó không xác định.

Ví dụ này cho thấy việc bỏ thông tin tọa độ tuyệt đối, chỉ giữ quan hệ giữa điểm với trọng tâm và quan hệ vòng quanh trọng tâm, giúp hai phép tịnh tiến và quay trở nên dễ xử lý.

5 Tổng kết

Tư tưởng tương tự là một dạng chuyển hóa. Điểm nổi bật của nó không phải là quy nạp hay suy diễn tuyến tính, mà là so sánh nhảy vọt giữa hai đối tượng có cấu trúc giống nhau. Vì vậy nó khó nắm chắc, nhưng khi dùng đúng thì rất mạnh.

Một phép tương tự tốt thường có ba đặc điểm:

- **Có cơ sở tương tự:** bài toán gốc và đối tượng được chuyển tới thật sự giống nhau ở những tính chất quan trọng.
- **Có tác dụng đơn giản hóa:** bài toán sau chuyển hóa dễ nghĩ hơn, quen thuộc hơn, chuẩn hóa hơn, hoặc có nhiều công cụ hơn.
- **Có khả năng di chuyển lời giải:** lời giải của đối tượng tương tự phải gắn chặt với cách giải bài toán mới. Nếu đã chuyển tập điểm thành chuỗi vòng, ta phải nghĩ tới các thuật toán chuỗi như KMP.

Tương tự không phải công cụ vạn năng. Nó chỉ có ý nghĩa khi bài toán cần một sự chuyển hóa. Với những bài đã rõ mô hình và lời giải trực tiếp, ép dùng tương tự chỉ làm suy nghĩ rối hơn.

Lời cảm ơn

Tác giả cảm ơn cô Xiang Qizhong vì đã hướng dẫn và giúp đỡ trong quá trình viết bài; cảm ơn huấn luyện viên Liu Rujia vì những gợi ý và khai mở; đồng thời cảm ơn các bạn đã đọc bản thảo và góp ý.

References

- [1] Yuan Yinzong. *Vượt ải toán học: lĩnh hội tư tưởng và phương pháp toán*.

- [2] Liu Rujia, Huang Liang. *Nghệ thuật thuật toán và thi tin học*.
- [3] USACO Open 2005, bài *Peaks*.
- [4] USACO February 2005, bài *Secret*.
- [5] Polish Olympiad in Informatics 2005 Stage I, bài *PUN*.